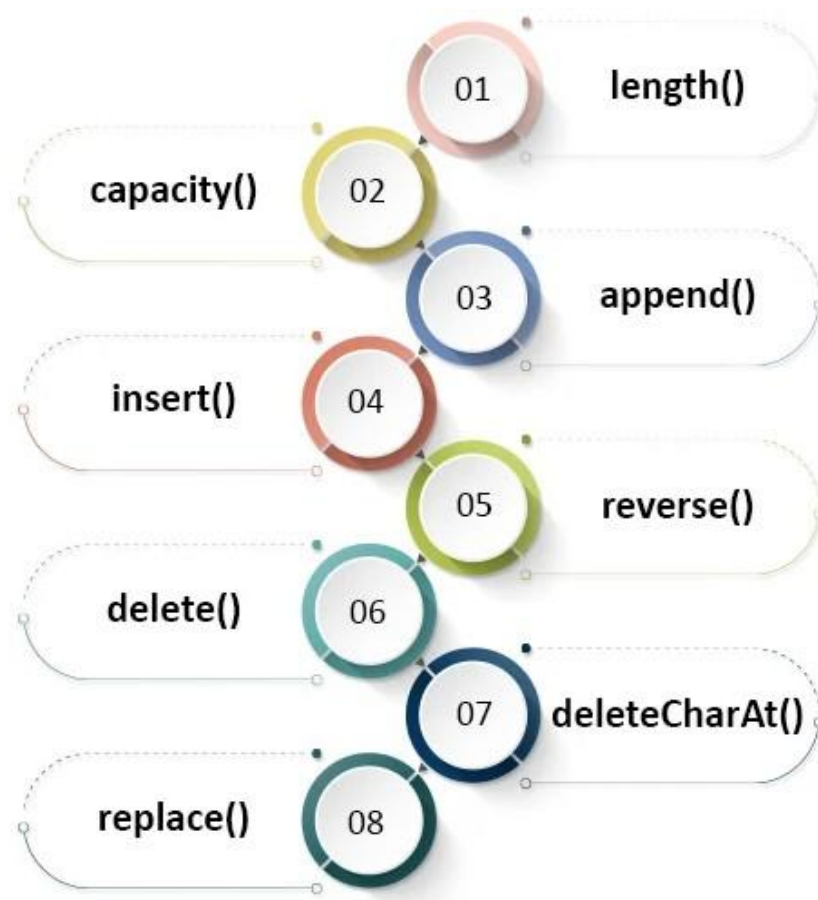


Java

Functions in String Builder & String Buffer :-

- StringBuffer and StringBuilder are mutable classes in Java used to create and manipulate dynamic strings.
- They provide many useful functions (methods) to modify strings efficiently without creating new objects each time.
- These functions allow us to append, insert, delete, replace, reverse, search, and manage capacity of strings.



- 1. append()** – Adds text at the end of the string.
`sb.append("World");`
- 2. indexOf()** -
`sb.indexOf("FCT");`
- 3. insert()** – Inserts text at a specified index.
`sb.insert(5, "Java");`

4. replace() – Replaces characters between given indexes.

```
sb.replace(2, 5, "XYZ");
```

5. delete() – Deletes characters between given indexes.

```
sb.delete(2, 5);
```

6. reverse() – Reverses the string.

```
sb.reverse();
```

7. charAt() – Returns character at given index.

```
sb.charAt(2);
```

8. substring() – Extracts a portion of the string.

```
sb.substring(2, 6);
```

9. repeat() – Repeats the given string multiple times.

```
sb.repeat("FCT",2);
```

1) String Builder –

Example -

```
package Basic_Concept;

public class stringbuilderdemo {
    public static void main(String[] args) {

        StringBuilder str1=new StringBuilder("Good");
        System.out.println("Before Append: "+str1);
        System.out.println("Using Append: "+str1.append(" Students"));
        System.out.println("After Append: "+str1);
        System.out.println("After Reverse: "+str1.reverse());
        System.out.println("Index Of: "+str1.indexOf("o"));
        System.out.println("Char At: "+str1.charAt(0));
        System.out.println("Index of Substring: "+str1.indexOf("t",3));
        System.out.println(str1.insert(0,"Sagar "));
        System.out.println(str1.delete(0, 7));
        System.out.println("String is: "+str1);
        System.out.println(str1.repeat(str1,8));
    }
}
```

Output –

Before Append: Good

Using Append: Good Students

After Append: Good Students

After Reverse: stnedutS dooG

Index Of: 10

Char At: s

2) String Buffer –

Example –

```
package Basic_Concept;

public class stringbufferdemo {

    public static void main(String[] args) {

        StringBuffer str2=new StringBuffer("Good");
        System.out.println("Before Append: "+str2);
        System.out.println("Using Append: "+str2.append(" Students"));
        System.out.println("After Append: "+str2);
        System.out.println("After Reverse: "+str2.reverse());
        System.out.println("Index Of: "+str2.indexOf("o"));
        System.out.println("Char At: "+str2.charAt(0));
        System.out.println("Index of Substring: "+str2.indexOf("t",3));
        System.out.println(str2.insert(0,"Sagar "));
        System.out.println(str2.delete(0, 7));
        System.out.println("String is: "+str2);
        System.out.println(str2.repeat(str2,8));

    }

}
```

Output –

Before Append: Good

Using Append: Good Students

After Append: Good Students

After Reverse: stnedutS dooG

Index Of: 10

Char At: s

Index of Substring: 6

Sagar stnedutS dooG

tnedutS dooG

String is: tnedutS dooG

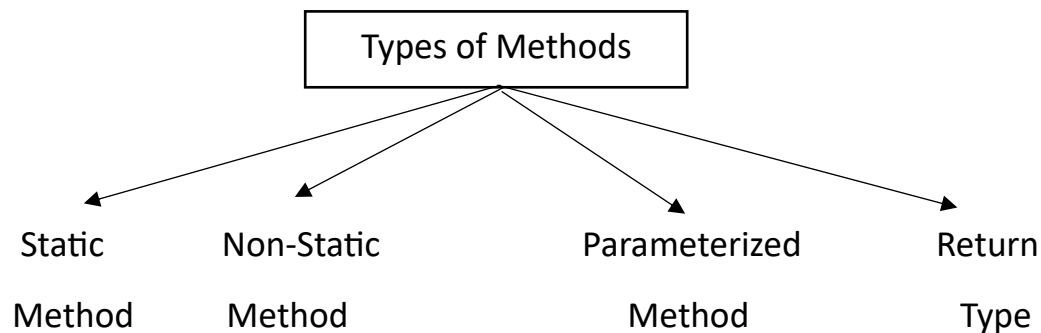
tnedutS dooGtnedutS dooGtnedutS dooGtnedutS dooGtnedutS dooGtnedutS dooGtnedutS dooGtnedutS dooG

Difference Between String, StringBuilder & StringBuffer :-

Features	String	StringBuilder	StringBuffer
Introduction	Introduced in JDK 1.0	Introduced in JDK 1.5	Introduced in JDK 1.0
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread Safe	Not Thread Safe	Thread Safe
Memory Efficiency	High	Efficient	Less Efficient
Performance	Can be slower for many concatenations	High(No-Synchronization)	Low(Due to Synchronization)
Usage	This is used when we want immutability.	This is used when Thread safety is not required.	This is used when Thread safety is required.

Methods in Java :-

- A method in Java is a block of code that performs a specific task.
- Methods are used to re-use code, improve readability, and reduce duplication.
- A method is executed only when it is called.
- Every method has a name, return type, parameters (optional), and a body.
- Methods can be predefined (built-in) like `println()`, or user-defined created by programmers.



Types of Methods :-

1. Static Method –

- A static method belongs to the class, not to objects.
- It can be called directly using the class name.
- No object creation is required.

Syntax –

```
static returnType methodName()  
{  
    //Block of Code  
}
```

Example –

```
package Basic_Concept;  
  
public class methodtypes {  
  
    public static void main(String[] args) {  
        display();        //Function Call  
    }  
}
```

```
static void display()    // Function Define
{
    System.out.println("Static method");
}
}
```

Output –

Static method

Task 1 – Find character at index '3' without using CharAt().

```
package assignment;

import java.util.Scanner;

public class string_task1 {

    public static void main(String[] args) {
        //Find character at index 5

        System.out.println("Enter the String:");
        Scanner sc=new Scanner(System.in);
        String s=sc.next();

        System.out.println("Enter the index:");
        int index=sc.nextInt();

        char ch[]=s.toCharArray();
        for(int i=0;i<ch.length;i++)
        {
            if(index==i)
            {
                System.out.println(ch[i]);
            }
        }
    }
}
```

Output –

Enter the String:

Cravita

Enter the index:

3

v

|

Task 2 – Find index at character 'v' without using indexOf().

```
package assignment;

public class string_task2 {

    public static void main(String[] args) {
        String s="Cravita";
        char ch='v';

        char arr[]=s.toCharArray();
        for(int i=0;i<arr.length;i++)
        {
            if(arr[i]==ch)
            {
                System.out.println(i);
            }
        }
    }
}
```

Output –

3